# Data Contract Validation Platform

Project Overview · Technologies · Page Guide

A working data-engineering platform that checks producer data batches against versioned data contracts, detects breaking and non-breaking schema changes, quarantines invalid records with reasons, publishes valid records exactly once, and reports measurable pipeline metrics.

| Item | Value |
| --- | --- |
| **Stack** | Python · FastAPI · SQLAlchemy · Pydantic · pandas · MySQL 8 · HTML/CSS/JavaScript · pytest · Ollama LLM |
| **Status** | Complete: 82 automated tests passing; 100,000-record benchmark at ~3,560 records/second |
| **Run** | http://127.0.0.1:8000 (dashboard) · http://127.0.0.1:8000/docs (API) |
| **Document date** | 10 November 2026 |

# Contents

1. What the project is (in simple words)
2. What we needed to build (requirements)
3. How the pipeline works, step by step
4. Technologies used and why
5. Project structure
6. Database tables
7. Explanation of each page
8. Demo walkthrough
9. Testing and performance results
10. How to run the project
11. Known limitations and future improvements
12. Glossary

# 1. What the project is

Companies move data between teams. A **producer** (for example, the order service) sends data files, and **consumers** (reports, finance, machine learning) read them. Problems start when the producer changes something without warning: a column disappears, a number becomes text, a new status value appears, or duplicate and broken rows slip in. The consumers then break, often silently.

A **data contract** is a written, machine-checkable promise about what the data must look like: which columns exist, their data types, whether they can be empty, which values are allowed, what the unique key is, which other dataset they reference, and which business rules must hold.

This platform enforces that promise at the entry point. For every incoming file it:

- **Compares the file's structure with the contract** and detects schema changes (missing, extra or renamed columns, type changes and so on).
- **Classifies each change** as **BREAKING** (consumers would fail, so the whole batch is blocked) or **NON-BREAKING** (safe to continue).
- **Validates every single record** (empty values, wrong types, invalid values, duplicates, broken references, business rules).
- **Quarantines invalid records**, keeping the original row, its row number, its source file and every reason it was rejected.
- **Publishes valid records** into the target table exactly once, even if the same file is sent again.
- **Records every pipeline run** with counts, timings and status, and shows them on a dashboard.
- **Explains results in plain language** using an AI model. The AI only explains; it never decides pass or fail.

## Demo business domain

An e-commerce example with two related datasets: **customers** (the parent) and **orders** (the child). Every order must belong to a published customer, which demonstrates referential integrity. Example rules: quantity between 1 and 1000, *total_amount* = *quantity* × *unit_price*, an order cannot ship before it was placed, and order dates cannot be in the future.

# 2. What we needed to build

The hackathon brief defined ten functional modules. The table shows each requirement and how it is delivered.

| Module | Requirement | How it is delivered |
|---|---|---|
| **A. Contracts** | Define columns, types, nullability, allowed values, keys, uniqueness, foreign keys, defaults, rules; store in MySQL; versioning | Contracts page and API. Every edit creates a new immutable version; old versions are never overwritten. |
| **B. Ingestion** | Upload CSV/JSON, capture file name, checksum, source system, time, run ID, contract version | Upload page; SHA-256 checksum; the original file is stored for traceability; bad files rejected with clear errors. |
| **C. Schema validation** | Detect missing/unexpected columns, type and nullability differences, key changes, renames, allowed-value changes | Structured schema report for every run (Validation Results page). |
| **D. Breaking vs non-breaking** | Explicit compatibility rules; never publish after a breaking change | Rule table with BACKWARD and STRICT policies; breaking batches are FAILED with nothing published. |

| Module | Requirement | How it is delivered |
|---|---|---|
| **E. Record validation** | Nulls, types, allowed values, duplicates, ranges, timestamps, references, business rules; multiple errors per record | 13 error codes, all computed in Python; every error stored with a reason. |
| **F. Transformation** | Deterministic cleaning without turning invalid data into valid data | Trimming, ISO-8601 timestamps, safe type conversion, declared normalisation, documented defaults. |
| **G. Publishing** | Only valid records, transaction-safe, idempotent | Deterministic record keys, unique constraint, single transaction, upsert. |
| **H. Quarantine** | List, filter by run or error type, view reasons, export CSV | Rejected Records page and API. |
| **I. Run tracking** | Run ID, counts, timings, status, failure reason | pipeline_runs table and Pipeline Runs page. |
| **J. Observability** | Totals, success %, durations, breaking/non-breaking counts, failed runs | Dashboard (live from the database) and JSON structured logs. |

### Additional requirements

- **AI assistant** using a configurable LLM (base URL, key, model), with a mock fallback when it is unavailable.
- **Sample data and generator** with a fixed random seed: a 100-row sample plus a generator for 100,000+ rows.
- **Automated tests** for 15 required scenarios, run against an isolated test database.
- **Idempotency and recovery**: safe retries, no duplicate records, no partial success.
- **Benchmark** with throughput measurement and at least one optimisation.
- **Documentation**: README, data dictionary, design decisions, performance report, test evidence.
- **Security**: secrets in .env only, file type and size checks, no SQL string building, no stack traces in responses.

## 3. How the pipeline works

Every uploaded file becomes a **pipeline run** that moves through these steps:

| Step | What happens | If something is wrong |
|---|---|---|
| **1. Upload** | File type (.csv/.json) and size are checked; SHA-256 checksum computed; original file saved. | HTTP 400 / 413 error immediately. |
| **2. Duplicate check** | Same checksum + same contract version already processed successfully? | Run marked SKIPPED_DUPLICATE; nothing republished. |
| **3. Parse** | File read as text; the exact original values are kept for traceability. | Empty or unreadable file: run FAILED with a reason. Single broken rows are quarantined as MALFORMED_ROW. |
| **4. Schema validation** | Columns and column-level types compared with the contract; each change classified. | Any BREAKING change: run FAILED, no records published. |
| **5. Transformation** | Trim text, standardise timestamps to ISO-8601, convert types, apply declared defaults. | Values that cannot be converted are flagged, never 'fixed'. |
| **6. Record validation** | Every rule checked on every row; a row can collect several errors. | Invalid rows go to quarantine with all reasons. |

| Step | What happens | If something is wrong |
|---|---|---|
| **7. Quarantine + publish** | Invalid rows stored in rejected_records; valid rows upserted into valid_records. | Any error rolls back the whole step; run FAILED; safe to retry. |
| **8. Finish** | Counts, duration and status saved. | – |

### Run statuses

| Status | Meaning |
|---|---|
| **PENDING** | Run created, waiting to be processed. |
| **RUNNING** | Currently being processed. |
| **COMPLETED** | All records valid and published. |
| **COMPLETED_WITH_ERRORS** | Valid records published, invalid records quarantined. |
| **FAILED** | File unreadable, breaking schema change, or an error that was rolled back. Nothing published. |
| **SKIPPED_DUPLICATE** | The exact same batch was already processed for this contract version. |

### Idempotency in simple words

*Idempotent* means 'doing it twice gives the same result as doing it once'. Each record gets a fixed identity computed from its contract and primary key (a **record key**). The database refuses two rows with the same record key, so sending the same order again **updates** it instead of creating a copy. Each record also has a **hash** of its content: if nothing changed, nothing is written. For late-arriving data, an older event can never overwrite a newer one because the event time is compared before updating.

## 4. Technologies used and why

| Technology | Used for | Why it was chosen |
|---|---|---|
| **Python 3.10** | All backend logic, scripts and tests | Standard language for data engineering; rich data libraries. |
| **FastAPI** | REST API (web server for /api/...) | Fast, modern, automatic API documentation at /docs, built-in request validation with Pydantic. |
| **Uvicorn** | Runs the FastAPI application | Lightweight, production-grade ASGI server. |
| **SQLAlchemy 2** | Database models, queries, transactions | Safe parameterised SQL (prevents SQL injection), clear table definitions, transaction control. |
| **Pydantic 2** | Validating API inputs/outputs and contract definitions; validating AI output | Catches bad input early with clear error messages. |
| **pydantic-settings / python-dotenv** | Reading configuration from .env | Keeps passwords and keys out of the code. |
| **MySQL 8 + PyMySQL** | Storing contracts, runs, published and rejected records | Required by the brief; supports transactions, unique constraints, foreign keys and JSON columns. |
| **pandas / NumPy** | Parsing, transforming and validating data | Vectorised column operations: whole columns are checked at once, which is what makes 100,000 records fast. |

| Technology | Used for | Why it was chosen |
|---|---|---|
| **HTML5, CSS3, vanilla JavaScript** | The dashboard (7 pages) | Required by the brief; no framework or build step needed. |
| **Fetch API** | Frontend to backend communication | Every number on screen comes from the backend; nothing hard-coded. |
| **SVG** | Dashboard charts | Drawn directly in the browser; no external chart library. |
| **Ollama (llama3.2:3b)** | Local AI model for explanations | Free, runs offline; uses an OpenAI-compatible API, so the company LLM can be configured later without code changes. |
| **httpx** | Calling the LLM API; test client | Modern HTTP client with timeouts. |
| **pytest + FastAPI TestClient** | 82 automated tests | Standard Python testing; TestClient calls the API without a running server. |
| **ReportLab** | Generating this PDF | Python PDF library (documentation only). |

Design principle: **deterministic rules decide, AI explains.** All pass/fail decisions are made by Python code that always gives the same answer for the same input. The AI only turns those results into readable explanations and suggestions, and its output is validated before it is shown.

# 5. Project structure

```
data contract validation platform/
  backend/
    main.py              FastAPI app, error handling, serves the dashboard
    config.py            settings from .env
    database.py          MySQL connection and sessions
    models/              database tables (SQLAlchemy)
    schemas/             request/response formats (Pydantic)
    routers/             API endpoints: contracts, ingestion, pipeline, validation, ai
    services/            business logic:
      ingestion_service        file checks, checksum, parsing
      schema_validator         batch schema vs contract
      compatibility_service    breaking / non-breaking rules
      transformation_service   cleaning and type conversion
      data_validator           record-level rules (13 error codes)
      quarantine_service       store / filter / export rejected records
      publishing_service       idempotent publishing
      pipeline_service         runs all steps in order, transactions
      run_service              reports and dashboard metrics
      llm_service              AI explanations (real LLM or mock)
    utils/               logging, IDs and hashing
  frontend/              index, contracts, ingest, validation, rejected, runs, ai pages + css/js
  tests/                 82 automated tests
  scripts/               init_db, generate_data, seed_demo, benchmark
  data/samples/          sample contracts, sample files, manifest, sample outputs
  docs/                  performance report, test results, this PDF
  README.md, DATA_DICTIONARY.md, DESIGN_DECISIONS.md, requirements.txt, .env.example
```

**Why this layout:** routers only handle HTTP; all real logic lives in services. Each service has one job, which makes it easy to test and easy to explain.

# 6. Database tables

| Table | What it stores |
|---|---|
| **data_contracts** | One row per dataset contract (name, dataset, owner). |
| **contract_versions** | Immutable versions of a contract: compatibility policy, unique keys, business rules. |
| **contract_columns** | Column definitions per version: type, required, nullable, primary key, allowed values, default, range, pattern. |
| **contract_foreign_keys** | References to other datasets (e.g. orders.customer_id → customers). |
| **source_files** | Every distinct uploaded file: name, SHA-256 checksum, size, stored location. |
| **pipeline_runs** | Every execution: status, counts, duration, failure reason, timings. |
| **schema_changes** | Every detected change with old/new schema, severity, classification and explanation. |
| **valid_records** | Published data, one row per record key (no duplicates possible). |
| **rejected_records** | Quarantined rows exactly as received, with source file and row number. |
| **rejection_errors** | One row per error of a rejected record (makes filtering by error type fast). |

Full column-level detail is in DATA_DICTIONARY.md.

# 7. Explanation of each page

Open **http://127.0.0.1:8000**. A dark sidebar on the left links to all seven pages. At the bottom of the sidebar, two status dots show whether the **database** and the **AI model** are reachable, plus a link to the API documentation.

## 7.1 Dashboard

**Purpose:** A live overview of the health of all pipelines. Every number is calculated from the database; it refreshes automatically every 15 seconds.

### What you see

- **KPI cards:** total runs, records processed, valid records, rejected records, validation success %, breaking changes, non-breaking changes, failed runs.
- **Valid vs rejected chart:** stacked bars for the most recent runs (grey bars = failed or skipped runs). Hover a bar for run details.
- **Top rejection reasons:** the most frequent error codes across all runs.
- **Run status** and **schema changes** charts.
- **Recent pipeline executions** table with links to each run's report.

### What you can do

- Click **Refresh** to reload immediately.
- Click a run ID to open its validation report.

**Backend APIs used:** `GET /api/metrics/summary, GET /api/health, GET /api/ai/status`

## 7.2 Contracts

**Purpose:** Create and manage data contracts and their versions.

### What you see

- Left: list of contracts with their latest version.
- Right: details of the selected contract with a **version selector** and five tabs:
- – **Columns**: name, type, required, nullable, primary key, allowed values, default, range, format, normalisation.
- – **Rules & keys**: primary key, unique business keys, foreign keys and business rules in readable form.
- – **Changes in vN**: what changed compared with the previous version, each marked BREAKING or NON-BREAKING.
- – **Published data**: the valid records currently in the target table, searchable by key.
- – **New version**: edit the contract to create the next version.

### What you can do

- **+ New contract**: opens a JSON editor; choose 'Sample: customers' or 'Sample: orders' to prefill. (Create customers before orders, because orders references customers.)
- **Check compatibility**: a dry run that shows how the proposed change would be classified, without saving.
- **Create version**: saves version N+1. If the change is breaking, you must tick 'I acknowledge breaking changes'; otherwise it is refused. Older versions are never modified.

**Backend APIs used:** `GET/POST /api/contracts, GET /api/contracts/{id}, POST /api/contracts/{id}/compatibility-check, POST /api/contracts/{id}/versions, GET /api/valid-records, GET /api/contracts/templates`

## 7.3 Data Ingestion

**Purpose:** Upload a producer batch and start the pipeline.

### What you see

- A form with: contract, contract version, source system name, a drag-and-drop file area and a 'Force reprocess' option.
- A **processing status** panel showing the run moving through Queued → Processing → Finished, with the file checksum.

### What you can do

- Choose a contract, drop a .csv or .json file (for example from data\samples\) and click **Start validation**.
- When it finishes you see the counts (input, valid, rejected, published, unchanged, stale, late) and buttons to open the validation report or the rejected records.
- **Force reprocess** re-runs a file that was already processed (useful for backfills); records are still not duplicated.

**Backend APIs used:** `POST /api/ingestion/upload, GET /api/runs/{run_id} (polled every second)`

## 7.4 Validation Results

**Purpose:** The full report for one pipeline run: what was wrong and why.

### What you see

- Run summary: status, counts, contract and version, source file, checksum, start/end time, duration, attempt number, per-stage timings and transformations applied (for example 'defaults applied: 1').
- **Schema validation report:** every detected schema change with its classification, severity and explanation.
- **Record-level errors:** bar chart of error codes per column (click one to see those records).
- **Error explanations:** plain description of each error code.
- **Sample rejected records:** click a row to see the original record and all reasons.

### What you can do

- Pick any run from the drop-down at the top.
- **Explain with AI** opens the AI Assistant for this run.
- **Retry run** (only for failed runs) starts a new attempt with the same file.

**Backend APIs used:** `GET /api/runs, GET /api/runs/{run_id}/report, POST /api/runs/{run_id}/retry`

## 7.5 Rejected Records (Quarantine)

**Purpose:** Browse every rejected record. Nothing is silently dropped.

### What you see

- Table: run ID, source file, original row number, error codes and the rejection reasons with the offending value.
- Pagination (50 rows per page).

### What you can do

- Filter by **run**, **error type** (e.g. FOREIGN_KEY_VIOLATION) and **contract**.
- Click a row to open the details: every error and the original record exactly as received.
- **Export CSV** downloads the records matching the current filters.

**Backend APIs used:** `GET /api/rejected, GET /api/rejected/{id}, GET /api/rejected/export, GET /api/error-codes`

### 7.6 Pipeline Runs

**Purpose:** The complete history of every pipeline execution.

#### What you see

- Run ID and attempt number, dataset and contract version, source file and source system, status, input / valid / rejected / published counts (and unchanged), schema changes (breaking count), duration, start time and failure details.

#### What you can do

- Filter by status or contract.
- **Retry** any FAILED run. This is safe because a failed run never published anything.
- Click a run ID to open its report.

**Backend APIs used:** `GET /api/runs, POST /api/runs/{run_id}/retry`

### 7.7 AI Assistant

**Purpose:** Plain-language explanations and recommendations for non-experts.

#### What you see

- The AI provider status (model name and whether it is reachable).
- A result panel with: the **authoritative facts** from the rule engine (status and counts), a risk level (computed by Python), a summary, key findings, recommended actions, which model produced it, and the exact facts that were sent to the model.

#### What you can do

- Choose a task: summarise a run, explain validation failures, explain schema changes, suggest corrective actions, or explain a contract version change.
- Choose the run (or contract and version) and click **Generate explanation**. A local model takes about 30–80 seconds on a laptop CPU.
- If the AI is unavailable or returns invalid output, a deterministic explanation is shown instead and clearly labelled.

**Backend APIs used:** `GET /api/ai/status, POST /api/ai/explain`

### 7.8 API documentation (/docs)

FastAPI automatically generates an interactive page at **http://127.0.0.1:8000/docs** listing every endpoint with its inputs and outputs. Each endpoint can be tried directly with *Try it out → Execute*. This is the easiest way to test the backend on its own.

## 8. Demo walkthrough

The script `scripts\seed_demo.py` performs these uploads through the real API. They can also be done by hand on the Data Ingestion page.

| # | Upload / action | Result | What it demonstrates |
|---|---|---|---|
| 1 | customers.csv (58 rows) | 50 published, 8 quarantined | Record validation and quarantine |

| # | Upload / action | Result | What it demonstrates |
|---|---|---|---|
| 2 | orders.csv (100 rows, shuffled) | 90 published, 10 quarantined | One of each defect: NULL, duplicate key, bad type, bad value, negative quantity, bad customer reference, bad date, wrong total, ship before order, duplicate event |
| 3 | orders.json | 0 new, 50 unchanged | JSON support and idempotency |
| 4 | orders_unexpected_column.csv | COMPLETED | Extra column = NON-BREAKING (ignored) |
| 5 | orders_missing_column.csv | FAILED, nothing published | Missing required column = BREAKING |
| 6 | orders_renamed_column.csv | FAILED | Rename without mapping detected = BREAKING |
| 7 | orders_late.csv | 10 inserted, 10 stale skipped | Late and out-of-order data never overwrite newer data |
| 8 | orders.csv again | SKIPPED_DUPLICATE | Duplicate batch protection |
| 9 | Orders contract v2 (+ optional column) | Created, NON-BREAKING | Contract versioning; v1 kept |

**Suggested presentation order:** Dashboard → Contracts (columns, rules, version changes) → Data Ingestion (upload orders_missing_column.csv live) → Validation Results (show the breaking change) → Rejected Records (filter and export) → Pipeline Runs (retry) → AI Assistant (start it early, since it takes about a minute).

# 9. Testing and performance results

## Automated tests

**82 tests pass** (about 40–65 seconds). They run against a separate database (`data_contract_test`) that is emptied before every test, so the demo data is never touched. The AI runs in mock mode during tests.

| Test file | What it covers |
|---|---|
| **test_contracts.py** | Create/read contracts, validation of contract definitions, versioning, breaking-change acknowledgement |
| **test_compatibility.py** | Every breaking / non-breaking rule (add, remove, rename, type, nullability, keys, enums, ranges) |
| **test_schema_validation.py** | Missing, unexpected and renamed columns; column type changes; STRICT policy |
| **test_data_validation.py** | Each error code, multiple errors per record, domain rules, timestamp standardisation |
| **test_pipeline.py** | Happy path, CSV and JSON, empty and malformed files, reports, metrics, CSV export |
| **test_idempotency.py** | Duplicate batch, forced rerun, partial failure + retry, late and out-of-order data |
| **test_generated_data.py** | Generator is reproducible; rejected rows equal the injected bad rows exactly |
| **test_ai.py** | AI fallback, invalid AI output, AI cannot change status, API key never exposed |

## Performance benchmark

| Records | Valid | Rejected | Matches injected errors | Time | Throughput |
|---|---|---|---|---|---|
| 1,000 | 950 | 50 | Yes | 0.41 s | 2,442 rec/s |
| 10,000 | 9,500 | 500 | Yes | 3.37 s | 2,965 rec/s |

| Records | Valid | Rejected | Matches injected errors | Time | Throughput |
|---|---|---|---|---|---|
| 100,000 | 95,000 | 5,000 | Yes | 28.09 s | 3,559 rec/s |

**Optimisation implemented:** each published record's JSON is now built once instead of twice (it was previously encoded once for the hash and again for storage). This made that stage about 17% faster and guarantees the stored data and its hash always match. Details are in docs/PERFORMANCE_REPORT.md.

# 10. How to run the project

**Prerequisites:** MySQL running (service MySQL80, port 3307 on this machine) and the project's .venv folder. Ollama is optional (only for real AI explanations).

### Start the application (backend and dashboard together)

```
cd "C:\Users\V.R.LAKSHMI PRASANNA\Videos\yottaflex-session-pipeline\data contract validation platform"
.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

Then open http://127.0.0.1:8000 (dashboard) or http://127.0.0.1:8000/docs (API). Stop with Ctrl+C.

### Other commands

| Purpose | Command |
|---|---|
| **Run all tests** | .venv\Scripts\python.exe -m pytest -v |
| **Load demo data (server running)** | .venv\Scripts\python.exe scripts\seed_demo.py |
| **Generate sample data** | .venv\Scripts\python.exe scripts\generate_data.py |
| **Generate 100,000 orders** | .venv\Scripts\python.exe scripts\generate_data.py --orders 100000 --customers 5000 --out data/generated |
| **Run benchmark** | .venv\Scripts\python.exe scripts\benchmark.py |
| **Create tables manually** | .venv\Scripts\python.exe scripts\init_db.py |
| **Check backend health** | Invoke-RestMethod http://127.0.0.1:8000/api/health |

**There is only one server.** The FastAPI backend serves both the API (/api/...) and the dashboard files (/ui/...). The dashboard's JavaScript calls the API using fetch().

### Configuration (.env)

| Variable | Purpose |
|---|---|
| **DATABASE_URL / TEST_DATABASE_URL** | MySQL connection for the app / for tests (test database name must contain 'test') |
| **LLM_PROVIDER** | openai_compatible (real LLM) or mock |
| **LLM_BASE_URL, LLM_API_KEY, LLM_MODEL** | LLM endpoint, key and model (Ollama: http://localhost:11434/v1, llama3.2:3b) |
| **LLM_TIMEOUT_SECONDS** | How long to wait for the AI (120) |
| **MAX_UPLOAD_MB, UPLOAD_DIR, LOG_LEVEL** | Upload limit, where original files are stored, logging detail |

# 11. Known limitations and future improvements

### Limitations

- Published data is stored in one shared table (JSON payload per record), not a separate typed table per dataset.
- Duplicate business events are detected within one batch, not across batches.
- Foreign keys must reference the parent's single-column primary key; parent data must be loaded first.
- Dates must be between the years 1677 and 2262; dates written with slashes are read day-first.
- Processing runs inside the web server; if the server crashes mid-run, that run stays RUNNING until retried (nothing partial is ever published).
- No login or user permissions.
- A small local AI model gives fairly generic advice; this is acceptable because it is advisory only.

### Future improvements

- Typed target tables per dataset, or a data-lake sink (Parquet / Iceberg).
- A job queue (Celery / RQ) with automatic recovery of interrupted runs.
- Database migrations with Alembic; authentication and per-producer permissions.
- Faster bulk loading (LOAD DATA into a staging table) and streaming for very large files.
- Approval workflow and consumer notifications for breaking contract versions.

## 12. Glossary

| Term | Meaning |
|---|---|
| **Data contract** | A machine-checkable agreement describing what a dataset must look like. |
| **Producer / consumer** | The system that sends data / the systems that use it. |
| **Batch** | One uploaded file of records. |
| **Schema** | The structure of the data: column names and types. |
| **Breaking change** | A change that would make existing consumers fail (e.g. a required column removed). |
| **Non-breaking change** | A safe change (e.g. a new optional column). |
| **Quarantine** | A separate store for rejected records, with the reasons they were rejected. |
| **Publishing** | Writing valid records into the target table that consumers read. |
| **Idempotent** | Running the same operation twice gives the same result as running it once. |
| **Checksum (SHA-256)** | A fingerprint of a file's content; identical files have identical checksums. |
| **Primary key** | The column(s) that uniquely identify a record (e.g. order_id). |
| **Foreign key / referential integrity** | A value that must exist in another dataset (an order's customer must exist). |
| **Late-arriving data** | Records that arrive after newer data for the same period was already published. |
| **Upsert** | Insert a record, or update it if it already exists. |
| **Transaction** | A group of database changes that are saved together or not at all. |